Cyber Physical Robotics

Dr. Paulo Garcia

2026

Cyber Physical Robotics

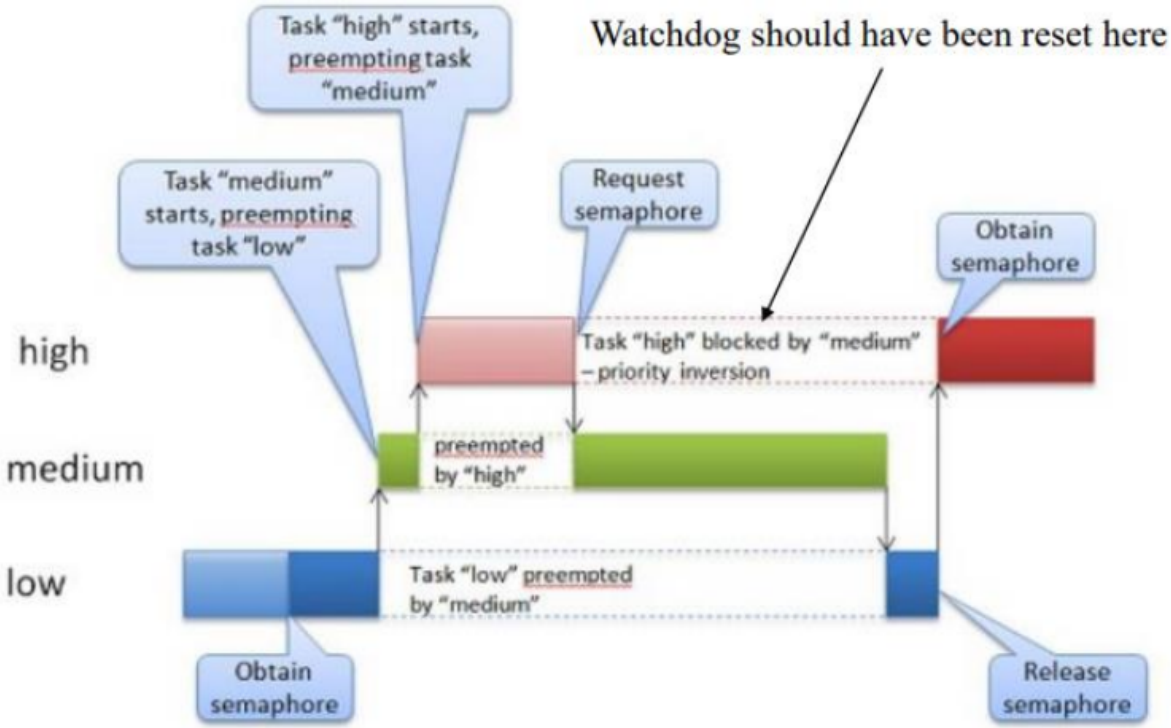
Today: Petri Nets

- A formalism for concurrent systems

Priority inversion

Case study: Mars Pathfinder

- Priority inversion bug prevented Watchdog update



Recall:

Priority inversion

A single system (robot, processor, etc.)

- Will always execute the ready task with highest priority
- If a system must execute N tasks:
  - How do we assign priorities such that all tasks meet their deadlines?

Priority inversion

A single system (robot, processor, etc.)

- Will always execute the ready task with highest priority
- If a system must execute N tasks:
  - If all tasks are periodic, **Rate Monotonic Scheduling**
    - Priorities are assigned statically
    - Proportional to the inverse of the period

Priority inversion

A single system (robot, processor, etc.)

- Will always execute the ready task with highest priority
- If a system must execute N tasks:
  - If all tasks are periodic, **Rate Monotonic Scheduling**
    - Priorities are assigned statically
    - Proportional to the inverse of the period
  - If at least one task is not periodic, **Earliest Deadline First**
    - Priorities are assigned dynamically
    - Proportional to the inverse of the deadline

CHULA ENG ACTNEW

Priority inversion

In both cases

- The problem is the **sharing of resources**
- Resource acquisition can break scheduling policy
  - We must use resource acquisition control (mutexes)
  - And appropriate priority policies (NPP, BIP, HLP)

Priority inversion

Now, consider K concurrent systems (2 or more robots, processors, etc.)

- Each always execute the ready task **assigned to it** with highest priority
- If the full system must execute N tasks:
  - Which tasks do we assign to each sub-system?
  - How do we assign priorities such that all tasks meet their deadlines?

Cyber Physical Robotics

How many solutions to:

- Assigning N tasks across K sub-systems? (where $K \leq N$)

How many solutions to:

- Assigning N tasks across K sub-systems? (where $K \leq N$)
- This is called the **Stirling number of the second kind**, given by:
  - $\sum_{i=0}^{K} \frac{(-1)^{K-i} i^N}{(K-i)!i!}$

Cyber Physical Robotics

n \ k	0	1	2	3	4	5	6	7	8	9	10
0	1										
1	0	1									
2	0	1	1								
3	0	1	3	1							
4	0	1	7	6	1						
5	0	1	15	25	10	1					
6	0	1	31	90	65	15	1				
7	0	1	63	301	350	140	21	1			
8	0	1	127	966	1701	1050	266	28	1		
9	0	1	255	3025	7770	6951	2646	462	36	1	
10	0	1	511	9330	34105	42525	22827	5880	750	45	1

CHULA ENG ACTNEW

Cyber Physical Robotics

2 main ways to solve this sort of problem

- Exact methods (mathematically): sometimes not possible
- Probabilistic methods (simulation): easier, not always correct

Exact solutions: **Integer Linear Programming**

- `https://en.wikipedia.org/wiki/Integer_programming`

Cyber Physical Robotics

Approximate solutions: concurrency models

- e.g., Petri Nets

Cyber Physical Robotics

- Quick revision of state machines

Cyber Physical Robotics

- A very simple robot with 2 arms



Cyber Physical Robotics

A very simple robot with 2 arms

- Sitting on a conveyor belt, waiting for pieces to arrive
- Has a work station next to it, can move pieces to workstation
- Next to it, shelf of tools it can access (soldering iron, grabber)

Cyber Physical Robotics

Workflow:

- Waits for (different) pieces to arrive
- Moves piece to workstation
- **If piece type A**: fetches soldering iron, solders
- **If piece type B**: fetches grabber, assembles components
- Moves piece back to conveyor belt

Cyber Physical Robotics

Workflow:

- Waits for (different) pieces to arrive
- Moves piece to workstation
- **If piece type A**: fetches soldering iron, solders
  - If component moves, fetches grabber to put it back in place
- **If piece type B**: fetches grabber, assembles components
  - If component breaks, fetches soldering iron to re-solder
- Moves piece back to conveyor belt

CHULA ENG ACTNEW

Cyber Physical Robotics

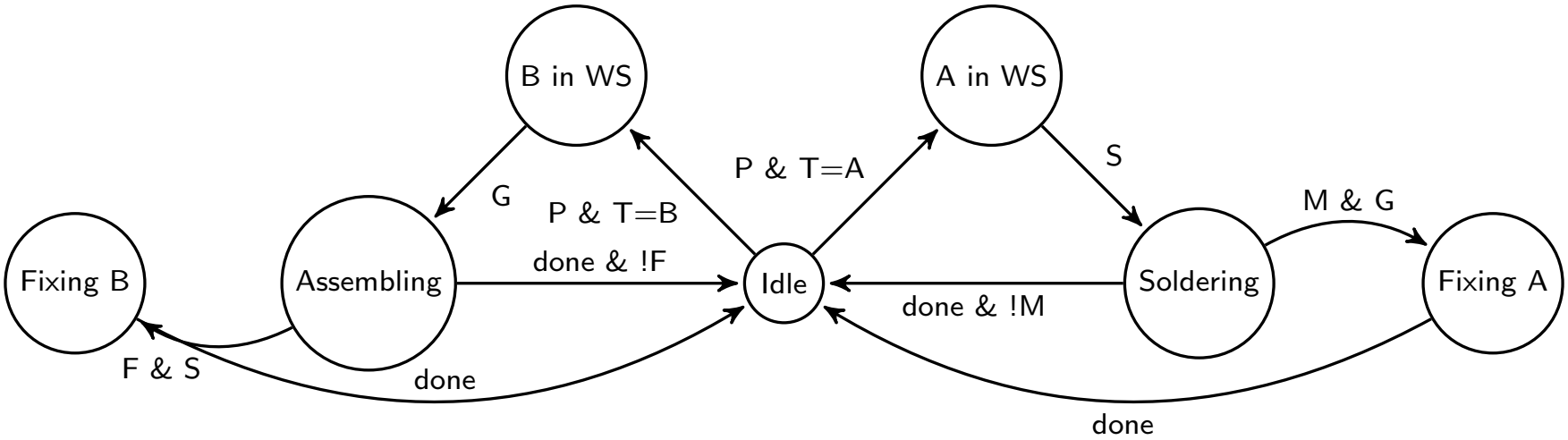
Sensors (robot inputs)

- P (set if piece arrives)
- T (type A or B)
- M (detects wrong movement when working on type A piece)
- F (detects faulty break when working on type B piece)
- S (soldering iron available)
- G (grabber available)

Cyber Physical Robotics

Let's implement robot behavior as a state machine

Robot state machine



Robot state machine

Why do we use state machines?

Robot state machine

Why do we use state machines?

- Higher level of abstraction than code/circuit
- Automatic verification
- Automatic code generation

Cyber Physical Robotics

Now: we're adding a second robot

- Trying to optimize throughput
- Sharing tools
- Do you see the problem?

Cyber Physical Robotics

- Robot 1 is in state "Soldering"
- Robot 2 is in state "Assembling"
- Both pieces have faults: system deadlock

Cyber Physical Robotics

Takeaway:

- Even if each individual system is correct ...
- ... there is potentially incorrect emergent behavior at scale

Cyber Physical Robotics

Takeaway:

- When designing distributed systems ...
- ... design the distributed parts together

Cyber Physical Robotics

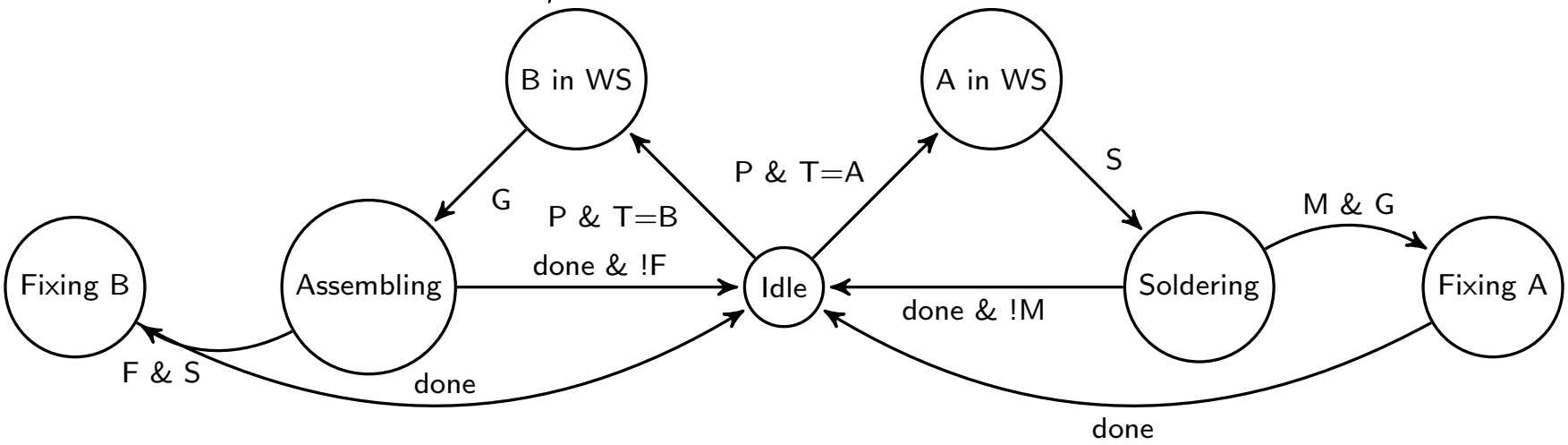
- State machines are not a good tool!
- (for distributed systems)
- Why?

Cyber Physical Robotics

- If sub-system 1 has N states
- If sub-system 2 has M states
- Combination has NxM states

Robot state machine

If we model 2 robots, each with this state machine... 49 states



Cyber Physical Robotics

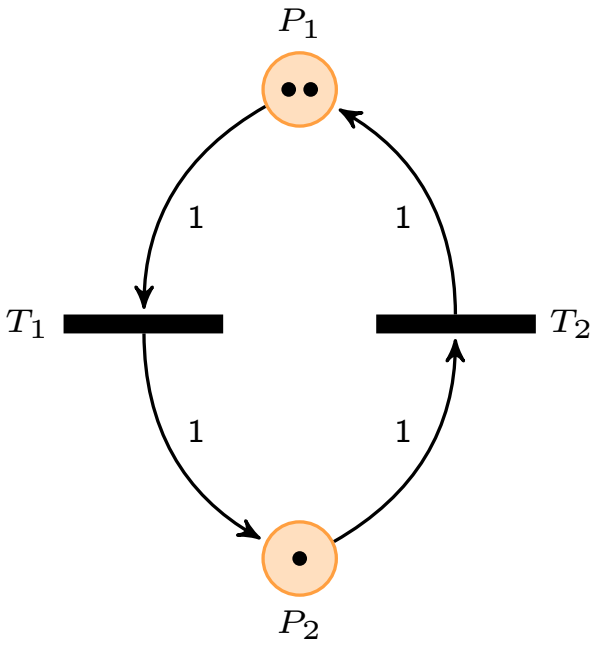
- Identifying (and preventing) troublesome states is not easy
- Generally, building the state machine is not feasible

Cyber Physical Robotics

Solution:

- A different formalism
- Better suited for distributed systems
- Petri Nets

Petri Nets



Petri Nets

Petri Nets

- Circles are called "places"
- Rectangles are called "transitions"
- Arrows are called "arcs"
- Black dots inside places are called "tokens"

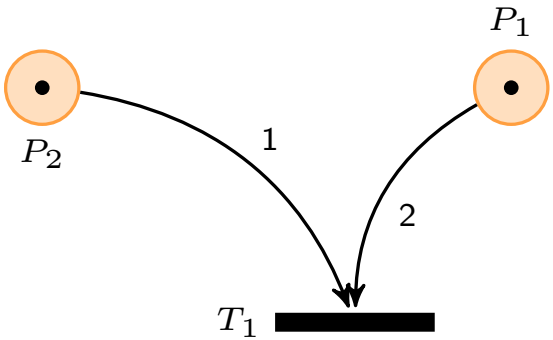
Petri Nets

Rules:

- A transition may "fire" if its input places have sufficient tokens

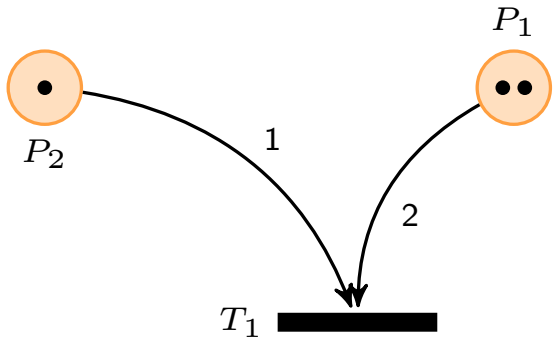
Petri Nets

Transition cannot fire



Petri Nets

Transition can fire

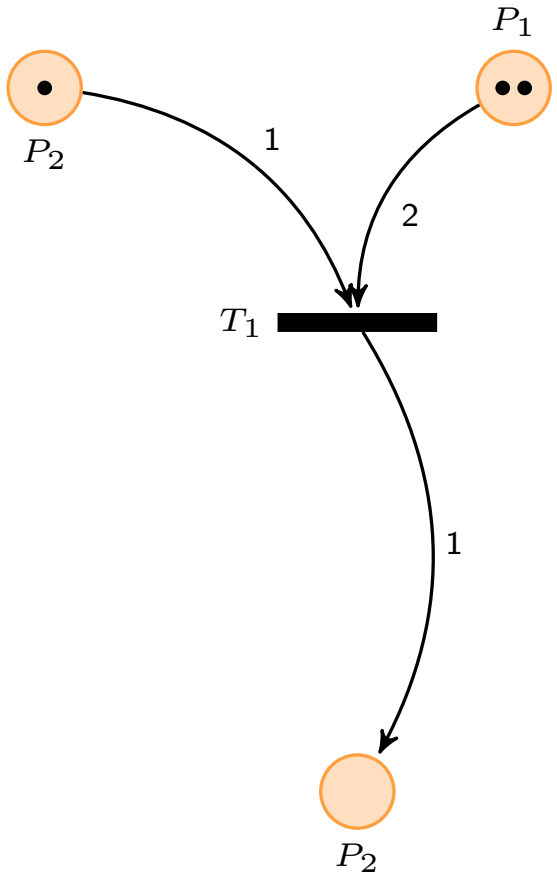


Petri Nets

Rules:

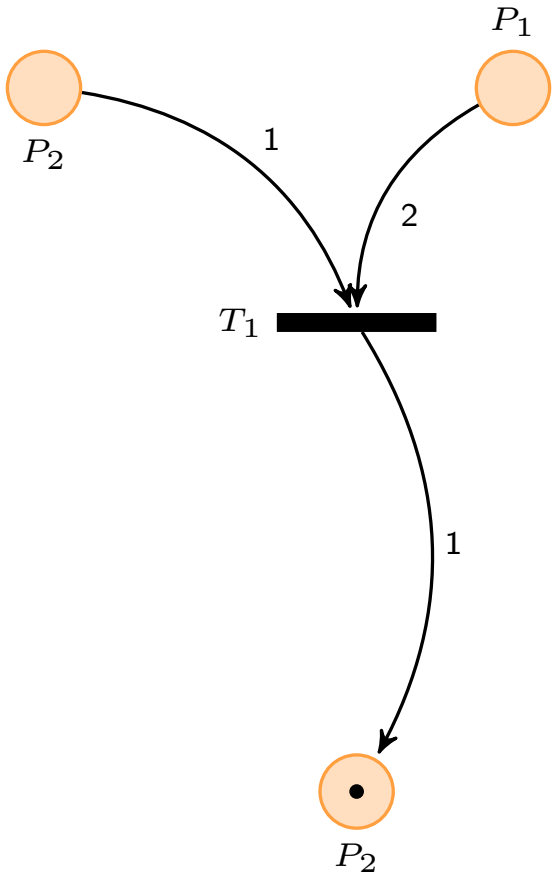
- When a transition fires, it consumes required input tokens
- And produces stated output tokens

Petri Nets



Before firing

Petri Nets



After firing

Petri Nets

Rules:

- If multiple transitions are enabled (able to fire)
- Which one fires is non-deterministic
- (thus, able to model concurrent systems)

Petri Nets

State machines

- In contrast, state machines are deterministic
- DFA: Deterministic Finite Automata
- Given a state and a set of inputs, only one state change is possible

Petri Nets

Petri Nets:

- Given a state (marking)
- Several next markings are possible

Models

- "All models are wrong, but some are useful"; George Box

Petri Nets as CPR Models

- Transitions model activity (work performed)
- Places model pre/post conditions for activity
- Tokens model availability of conditions

Cyber Physical Robotics

Workflow:

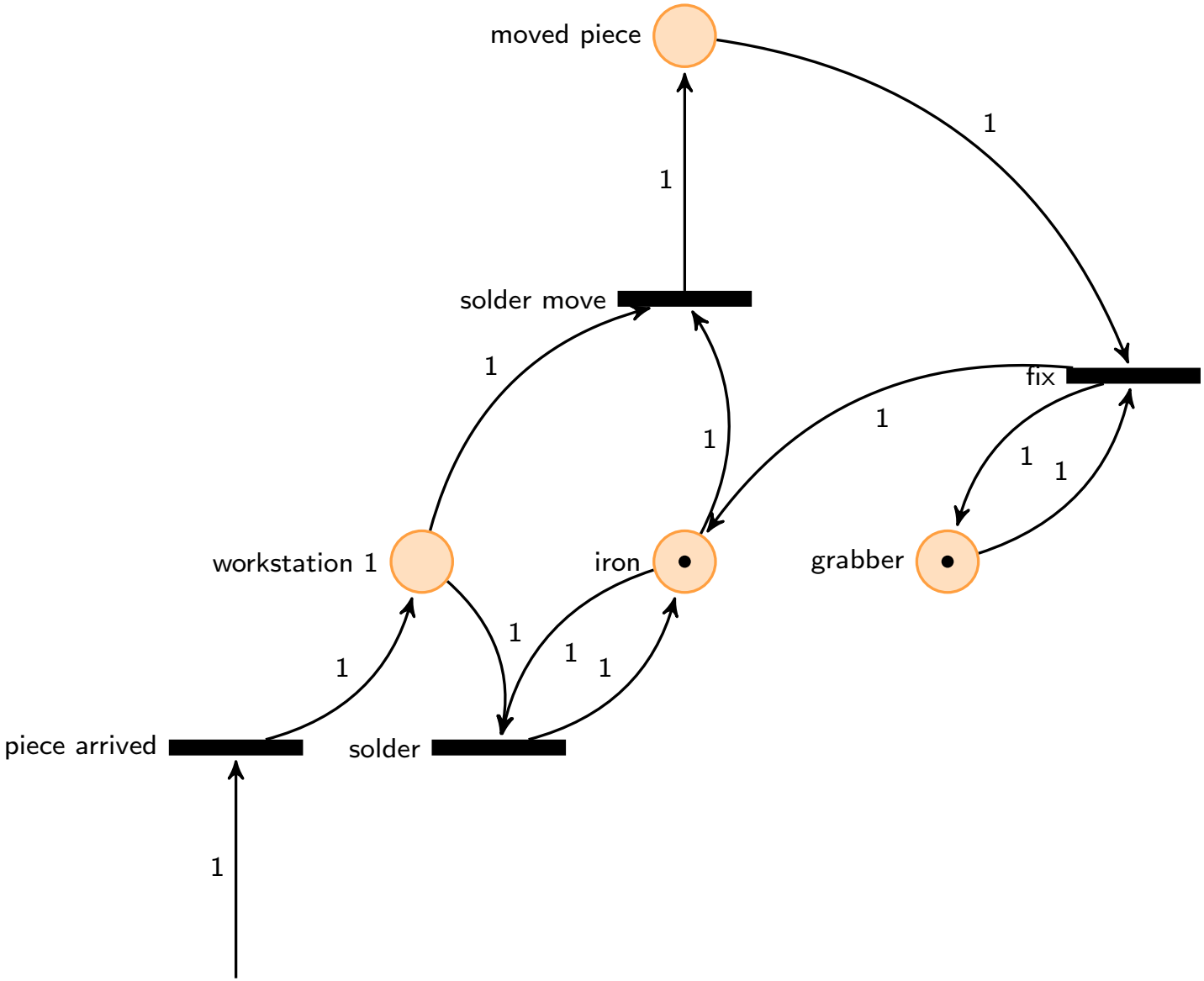
- Waits for (different) pieces to arrive
- Moves piece to workstation
- **If piece type A**: fetches soldering iron, solders
- **If piece type B**: fetches grabber, assembles components
- Moves piece back to conveyor belt

Cyber Physical Robotics

Workflow:

- Waits for (different) pieces to arrive
- Moves piece to workstation
- **If piece type A**: fetches soldering iron, solders
  - If component moves, fetches grabber to put it back in place
- **If piece type B**: fetches grabber, assembles components
- Moves piece back to conveyor belt
  - If component breaks, fetches soldering iron to re-solder

Petri Nets as CPR Models



Petri Nets as CPR Models

- Build the Petri Net for both robots

Petri Nets as CPR Models

Advantages

- Deadlock analysis
- Liveness analysis
- Reachability analysis
- Boundedness analysis

Petri Nets as CPR Models

Lots of Petri Net software

- PIPE (Petri Net modeler)
  `https://github.com/sarahtattersall/PIPE`
- PNet (Python library for Petri Net simulation)

Petri Nets as CPR Models

Advantages

- Graphical (visual) models are nice (for humans)
- But the big advantage is that Petri Nets are mathematically formalized
- (thus, computers can do all sorts of verifications automatically)

Petri Nets as CPR Models

A gentle introduction

- to-read
- `https://www.sciencedirect.com/science/article/pii/S1875389211007565`

Petri Nets (contd.)

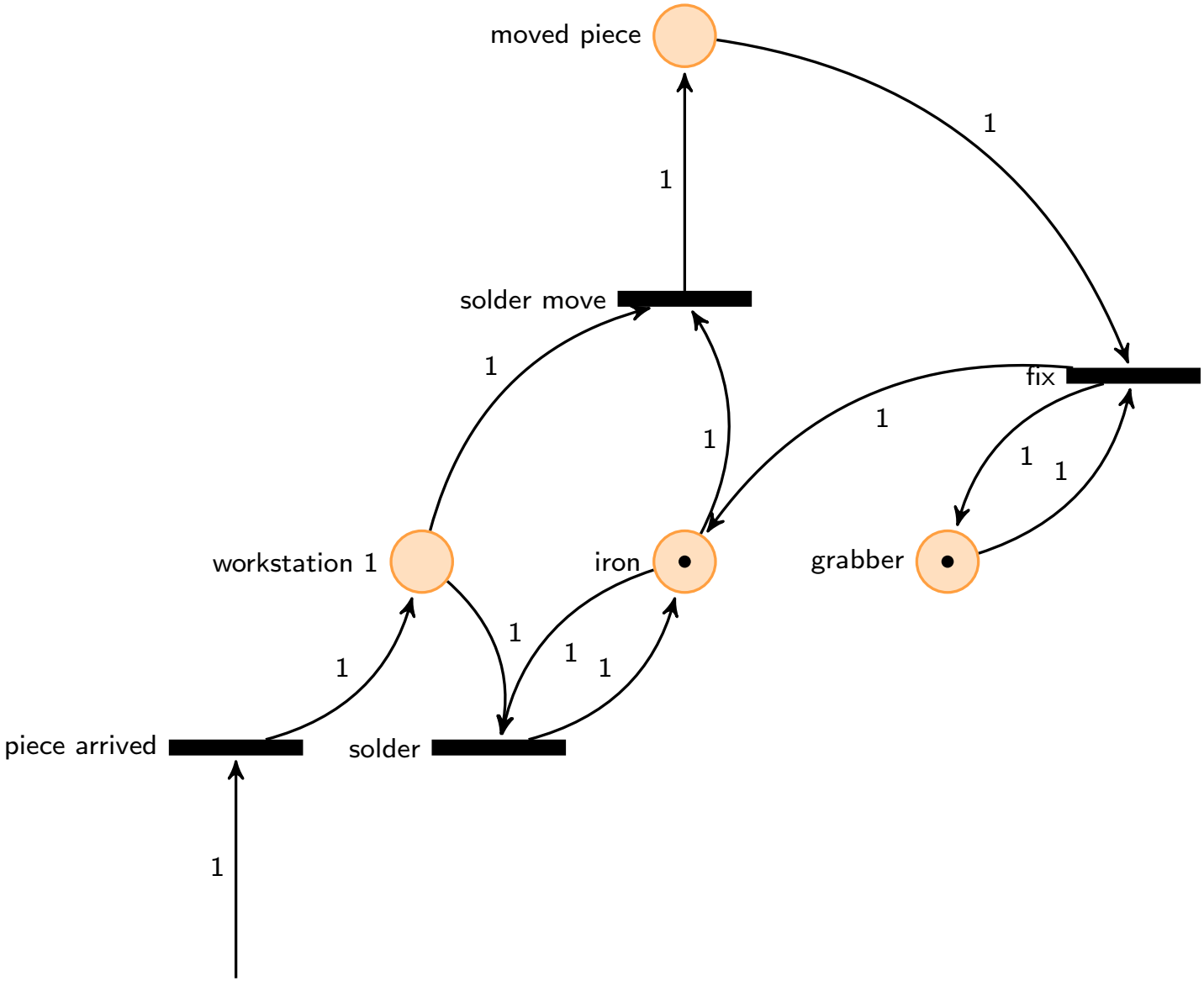
- Common patterns
- Mathematical formalizations of Petri Nets
- Liveness and deadlock analysis

Petri Nets

Common patterns: objectives

- Identify recurring use cases
- Gain deeper intuition for Petri Nets by looking at solutions for those use cases

Petri Nets as CPR Models



Petri Nets

Solution for 2 robots?

Petri Nets patterns

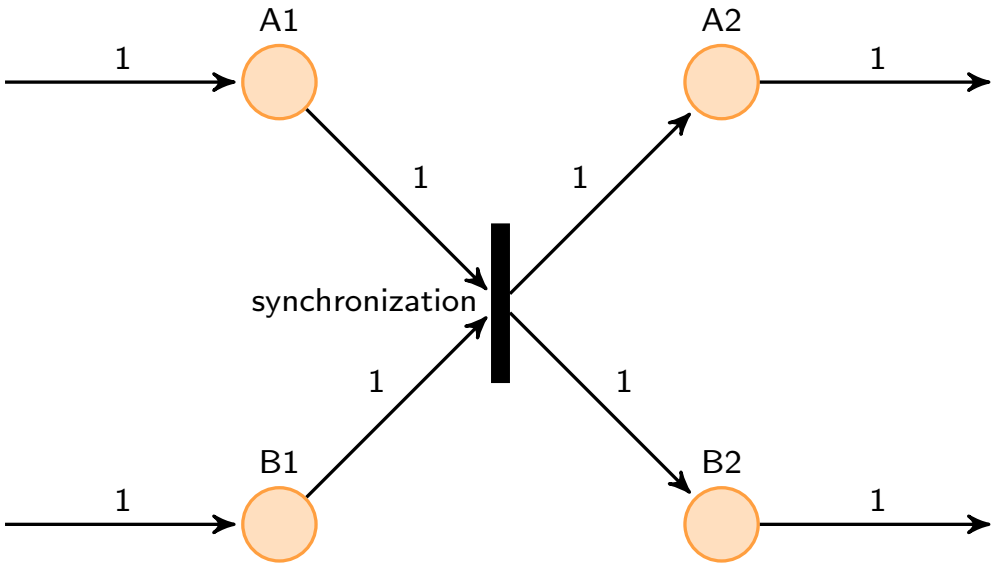
Problem:

- Synchronizing parallel processes

Example:

- Robot A and B can only begin certain operation when both have reached a certain stage

Petri Nets as CPR Models



Petri Nets patterns

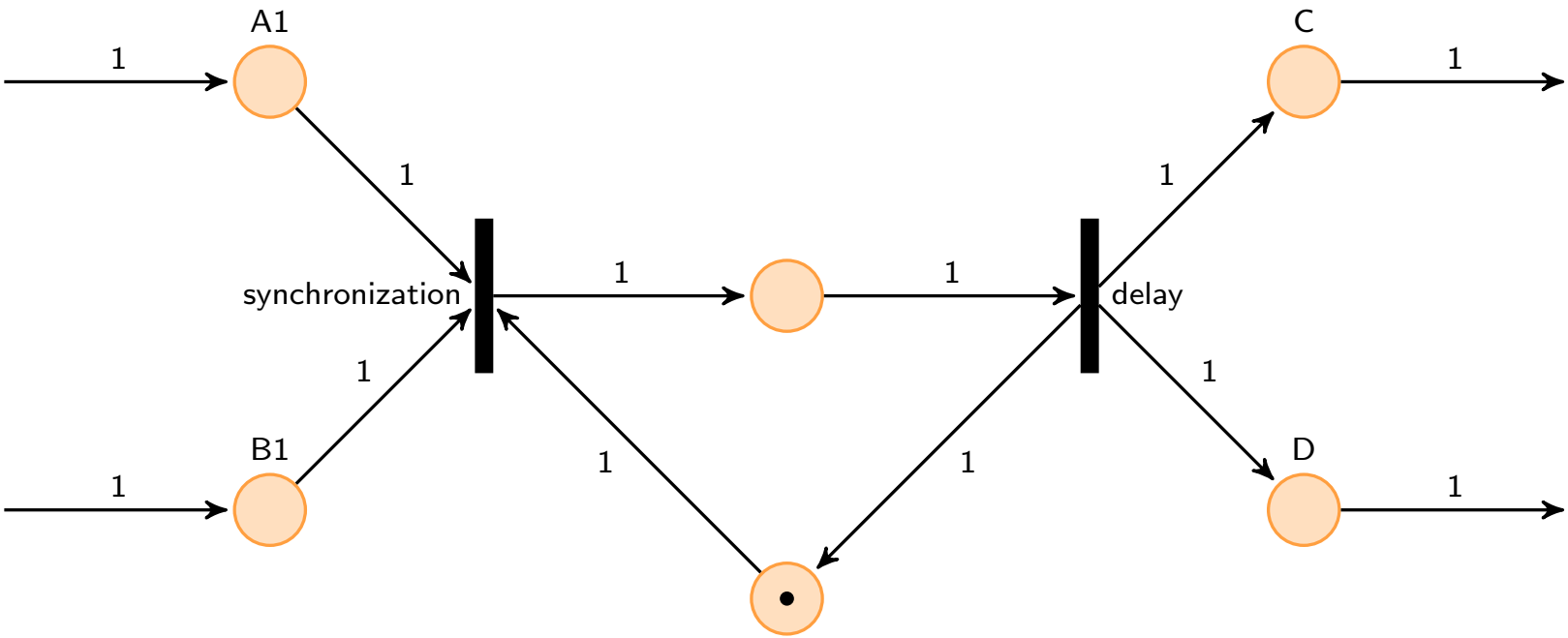
Problem:

- Adding a delay between two events

Example:

- Robot A and B are prevented from producing more quickly than C and D can consume

Petri Nets as CPR Models



Petri Nets patterns

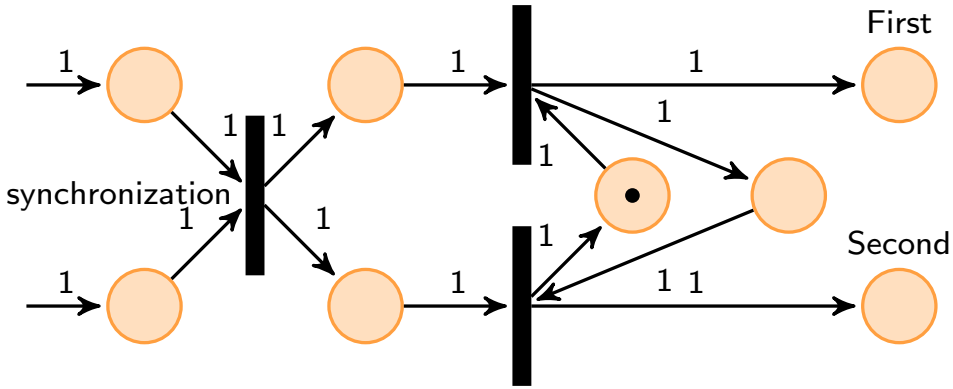
Problem:

- Creating strict alternation

Example:

- Robots A and B receive inputs in round-robin

Petri Nets as CPR Models



Petri Nets

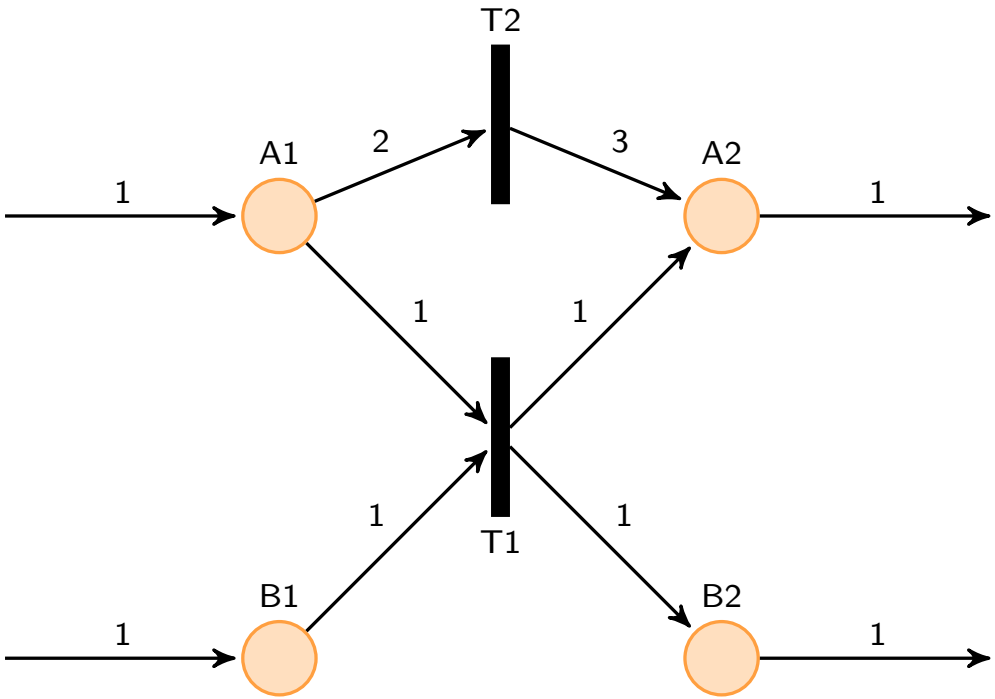
- Graphical representations are nice
- Mathematical models are nicer

Petri Nets Formalism

(**note**: many possible formalisms exist)

- The point is to use one that suits our needs

Petri Nets Formalism



Petri Nets Formalism

- You drew the network
- You believe it makes sense
- How would you simulate/implement it (in software)?

Petri Nets Formalism

We *could*

- Build a Petri Net class
- Places, Transitions.....
- Iterate over transitions and update places
- (code example)

Petri Nets Formalism

Alternative:

- Model the network mathematically
- Use one of the gazillion math libraries to implement it

Petri Nets Formalism

Take-home lesson:

- Equivalencies are your friends
- If two things are equivalent, you just need to implement one of them
- (and convert accordingly)

Petri Nets Formalism

A convenient formalism:

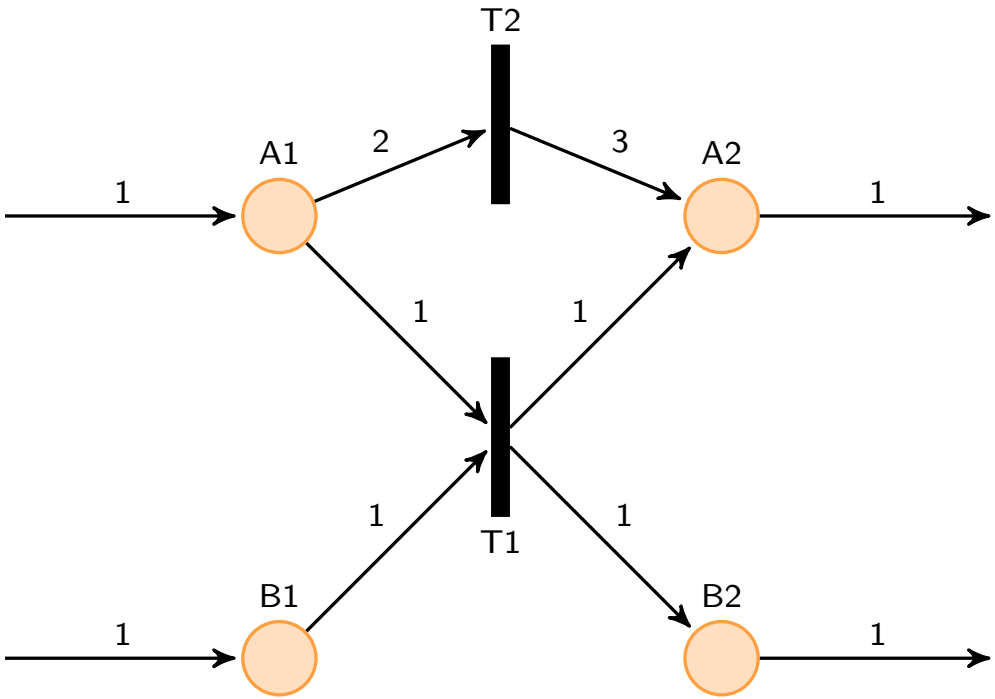
- Linear Algebra

Petri Nets Formalism

A convenient formalism: Linear Algebra

- A Petri Net is described by a matrix with:
  - p rows, representing places
  - t columns, representing transitions

Petri Nets Formalism



Petri Nets Formalism

A convenient formalism: Linear Algebra

- $P_n = \left(\begin{bmatrix} T1 & T2 & \\ -1 & -2 & A1 \\ 1 & 3 & A2 \\ -1 & 0 & B1 \\ 1 & 0 & B2 \end{bmatrix} \right)$
- Where column elements define what each transition firing does to each place

Petri Nets Formalism

A convenient formalism: Linear Algebra

- If marking vector $M = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}$
- Depicts number of tokens in each place

Petri Nets Formalism

A convenient formalism: Linear Algebra

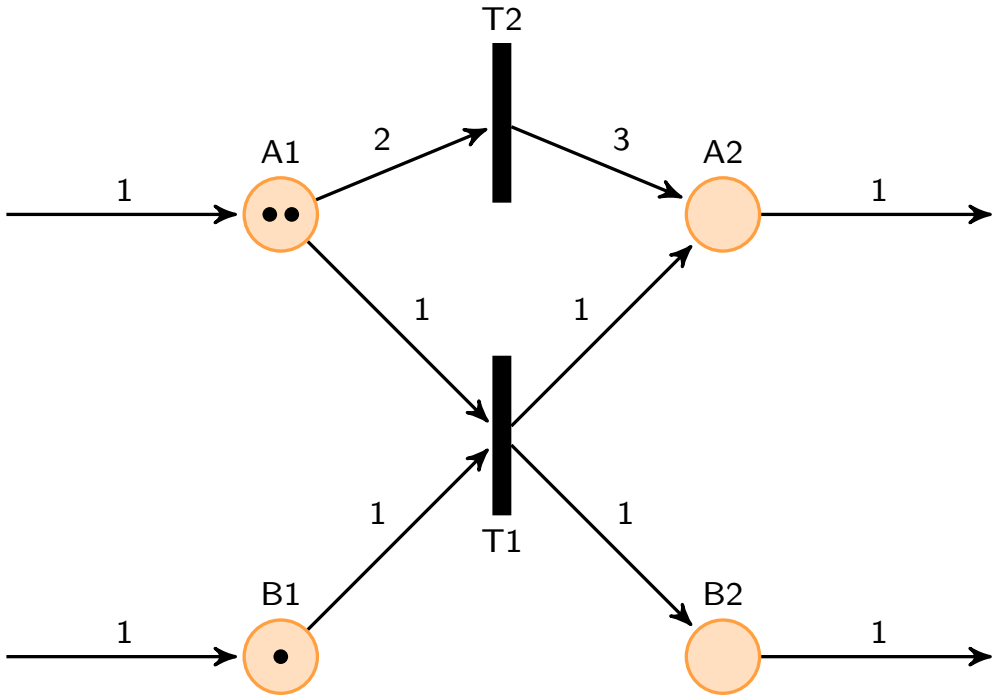
- If firing vector $F = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$
- Depicts firing of a transition

Petri Nets Formalism

A convenient formalism: Linear Algebra

- Then: $\mathbf{M_k} = \mathbf{P_n F} + \mathbf{M_{k-1}}$
- is the evolution of the network
- (assuming F represents valid transitions)

Petri Nets Formalism



Firing transition 2

Petri Nets Formalism

A convenient formalism: Linear Algebra

- Then: $\mathbf{M_k} = \mathbf{P_n F} + \mathbf{M_{k-1}}$
- $P_n = \begin{bmatrix} T1 & T2 & \\ -1 & -2 & A1 \\ 1 & 3 & A2 \\ -1 & 0 & B1 \\ 1 & 0 & B2 \end{bmatrix}$
- $F = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$
- $M_{k-1} = \begin{bmatrix} 2 \\ 0 \\ 1 \\ 0 \end{bmatrix}$

CHULA ENG

Petri Nets Formalism

A convenient formalism: Linear Algebra

- Then: $\mathbf{M_k} = \begin{bmatrix} 0 \\ 3 \\ 1 \\ 0 \end{bmatrix}$

Petri Nets Formalism

Summary:

- Petri Nets can model concurrency more efficiently than state machines
- You can design concurrent systems using Petri Nets
- They are equivalent to a linear algebra equation
- You just need to evaluate the equation to observe Petri Net behavior

CHULA ENG ACTNEW

Petri Nets Formalism

Advantages:

- Let's say we want to know what would happen to the network if T1 fired twice

Petri Nets Formalism

Advantages:

- Let's say we want to know what would happen to the network if T1 fired twice
- $\mathbf{M_2} = \mathbf{P_n F} + \mathbf{M_1}$
- $\mathbf{M_1} = \mathbf{P_n F} + \mathbf{M_0}$
- $F = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$

Petri Nets Formalism

Advantages:

- Let's say we want to know what would happen to the network if T1 fired twice
- $\mathbf{M_2} = \mathbf{P_n F} + \mathbf{P_n F} + \mathbf{M_0}$
- $F = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$

Petri Nets Formalism

Advantages:

- Let's say we want to know what would happen to the network if T1 fired twice
- $\mathbf{M_2} = \mathbf{P_n F_2} + \mathbf{M_0}$
- $F_2 = F + F = \begin{bmatrix} 0 \\ 2 \end{bmatrix}$

Petri Nets Formalism

Advantages:

- Let's say we want to know what would happen to the network if T1 fired twice
- $\mathbf{M_2} = \mathbf{P_n F_2} + \mathbf{M_0}$
- $F_2 = F + F = \begin{bmatrix} 0 \\ 2 \end{bmatrix}$
- (notice it also works for different transitions)

Petri Nets Formalism

Advantages:

- Let's say we want to know if a certain state (marking) of the network is possible
- We can brute-force it.....

Petri Nets Formalism

Advantages:

- Let's say we want to know if a certain state (marking M) of the network is possible
- We can brute-force it..... **or**:
- Is there any F_i, such that $\mathbf{M = P_n F_i + M_0}$
- where all the elements of F_i are positive integers?

Petri Nets Formalism

This is called the **reacheability problem**

- Is there any F_i, such that $\mathbf{M} = \mathbf{P_n F_i} + \mathbf{M_0}$
- where all the elements of F_i are positive integers?

Petri Nets Formalism

Does a network have useless loops?

- I.e., a set of transitions that takes us back to the same place we started?

Petri Nets Formalism

Does a network have useless loops?

- I.e., a set of transitions that takes us back to the same place we started?
- Yes, if $\exists F_i \rightarrow P_n F_i = 0$

CHULA ENG ACTNOW

Petri Nets Formalism

This only makes sense if the firing matrix is valid

- (i.e., that a transition has sufficient input tokens to fire)
- How to determine, from a marking matrix M_k and a network P_n, whether a given transition can fire?
- (for you to consider at home)

Next

- Let's look at Monte-Carlo methods
- Start with natural systems
- Understand the principles of Monte-Carlo simulation

Next

Afterwards:

- Monte-Carlo methods to trigger Petri-Nets
- (remember firing is non-deterministic)
- Making sense of complex artificial systems